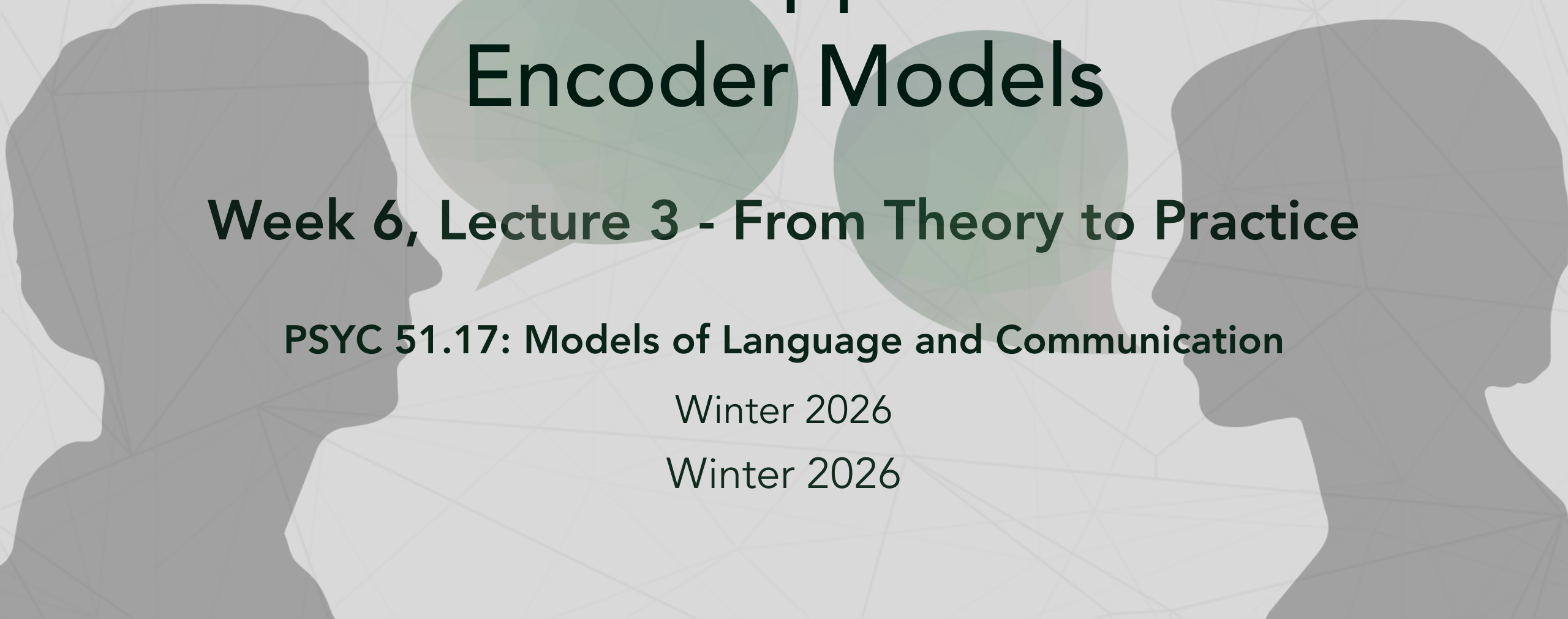
PSYC 51.17: Models of Language and Communication

Lecture 20: Applications of Encoder Models



Week 6, Lecture 3 - From Theory to Practice

PSYC 51.17: Models of Language and Communication

Winter 2026

Winter 2026

Today's Agenda

1. **Real-World Applications**: Where BERT shines
2. **Cognitive Neuroscience**: Brain-model parallels
3. **Understanding vs. Pattern Matching**: The big debate
4. **Limitations**: What BERT can't do
5. **Practical Tips**: Deployment and optimization
6. **Future Directions**: Where are we heading?

Goal: Connect BERT to real applications and understand broader implications

Winter 2026

BERT Applications

BERT excels at understanding tasks:

Classification Tasks:

- Sentiment Analysis
- Topic Classification
- Spam Detection
- Intent Recognition

Token-Level Tasks:

- Named Entity Recognition (NER)
- Part-of-Speech Tagging
- Word Sense Disambiguation

Span-Level Tasks:

- Question Answering
- Extractive Summarization
- Information Extraction

Sentence-Pair Tasks:

- Semantic Similarity
- Natural Language Inference
- Paraphrase Detection

Case Study: Google Search

BERT revolutionized search in 2019

```
# Why word order matters: BERT understands prepositions!
```

More Examples of Context-Sensitive Queries

Query	Before BERT	With BERT
"can you get medicine for someone pharmacy"	Generic pharmacy results	Picking up prescriptions for others
"do estheticians stand a lot at work"	Esthetician job listings	Physical demands of the job
"parking on a hill with no curb"	Parking tickets, curb info	How to park safely without a curb

Google reported BERT improved 1 in 10 searches in English

Question Answering with BERT

Extractive QA: Find answer span in passage

Example

Context: "The Normans (Norman: Nourmands; French: Normands; Latin: Normanni) were the people who in the 10th and 11th centuries gave their name to Normandy, a region in France."

Question: "In what country is Normandy located?"

Answer: France

```
from transformers import pipeline

# Load QA pipeline with BERT
qa_pipeline = pipeline("question-answering", model="bert-large-uncased-whole-word-masking-finetuned-squad")

# Ask question
```


continued...

Question Answering with BERT

```
)

print(result)
# {'answer': 'France', 'score': 0.987, 'start': 134, 'end': 140}
```

BERT predicts start and end positions of the answer span!

Winter 2026

...continued

Named Entity Recognition

Token-level classification task

Example

Input: "Apple Inc. is headquartered in Cupertino, California."

Output:

- Apple Inc. → {ORGANIZATION}
- Cupertino → {LOCATION}
- California → {LOCATION}

continued...

```
from transformers import pipeline
```

Named Entity Recognition

```
9
10 for entity in entities:
11  print(f"{entity['word']}: {entity['entity']} (score: {entity['score']:.2f})")
12
13 # Output:
14 # Apple: B-ORG (score: 0.99)
15 # Inc: I-ORG (score: 0.99)
16 # Cupertino: B-LOC (score: 0.99)
```

Winter 2026

...continued...

Named Entity Recognition

```
# California: B-LOC (score: 0.99)
```

Winter 2026

...continued

Sentiment Analysis

Sequence classification task

Examples

- "This movie was absolutely amazing!" → {POSITIVE}
- "The product broke after one week." → {NEGATIVE}
- "The weather is cloudy today." → {NEUTRAL}

```
from transformers import pipeline

# Load sentiment analysis pipeline
sentiment_pipeline = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")

# Analyze sentiments
```


continued...

Sentiment Analysis

```
 "The product broke after one week.",
 "The weather is cloudy today."
]

for text in texts:
 result = sentiment_pipeline(text)[0]
 print(f"{text}")
 print(f" → {result['label']} (confidence: {result['score']:.2f})\n")
```

Applications: Customer reviews, social media monitoring, brand sentiment

...continued

Semantic Similarity

Measuring sentence similarity with BERT embeddings

```
from transformers import BertTokenizer, BertModel
import torch
import torch.nn.functional as F

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertModel.from_pretrained('bert-base-uncased')

def get_sentence_embedding(sentence):
 inputs = tokenizer(sentence, return_tensors='pt', padding=True, truncation=True)
 outputs = model(**inputs)
 # Use [CLS] token embedding as sentence representation
 return outputs.last_hidden_state[:, 0, :]

# Compare sentences
```


continued...

Semantic Similarity

```
sent1 = "The cat is sleeping on the couch"
sent2 = "A feline is resting on the sofa"
sent3 = "The weather is nice today"

emb1 = get_sentence_embedding(sent1)
emb2 = get_sentence_embedding(sent2)
emb3 = get_sentence_embedding(sent3)

# Compute cosine similarities
sim_12 = F.cosine_similarity(emb1, emb2).item()
sim_13 = F.cosine_similarity(emb1, emb3).item()

print(f"Similarity (1-2): {sim_12:.3f}") # High (paraphrases)
print(f"Similarity (1-3): {sim_13:.3f}") # Low (different topics)
```


...continued

Cognitive Neuroscience Perspective

How do brains and models process language?

Predictive Processing in the Brain:

- Brain constantly predicts upcoming input
- N400: Neural response to unexpected words
- P600: Syntactic anomaly detection
- Context shapes predictions
- Prediction errors drive learning

Predictive Processing in Models:

- BERT: Predict masked words
- GPT: Predict next word
- Both use context to predict
- Surprise = high loss
- Gradient descent = learning

Similarities:

Prediction in Brains vs. Language Models

Parallels between neural and artificial systems

Phenomenon	Human Brain	Transformer Models
Surprise	N400 amplitude (EEG)	Cross-entropy loss
Hierarchy	sounds → words → sentences	tokens → phrases → meaning
Context	Prior discourse, world knowledge	Self-attention over sequence
Representation	Population coding (neurons)	Distributed embeddings (vectors)

```
# Concrete example: Surprise/N400 parallel
sentence_a = "I take my coffee with cream and sugar" # Expected
sentence_b = "I take my coffee with cream and socks" # Surprising

# Brain: N400 amplitude higher for "socks"
# Model: Higher loss for "socks"
loss_a = model.compute_loss("sugar", context) # Low loss
loss_b = model.compute_loss("socks", context) # High loss
```


continued...

Prediction in Brains vs. Language Models

```

# Both systems encode "surprisal" = -log P(word | context)
surprisal = -np.log(model.predict_prob("socks", context))
# Correlates with N400 amplitude in EEG studies!
```

Question: Are these superficial analogies or deep connections?

Reference: Kuperberg & Jaeger (2016) - "What do we mean by prediction in language comprehension?"

...continued

Neural Encoding with Language Models

Can we predict brain activity from language models?

```
# Neural encoding experiment workflow
import numpy as np
from transformers import BertModel

# 1. Participant reads sentences while in fMRI scanner
sentences = ["The dog chased the cat", "She opened the door", ...]
brain_activity = fmri_scanner.record(sentences) # (n_sentences, n_voxels)

# 2. Extract BERT representations for same sentences
bert = BertModel.from_pretrained("bert-base-uncased")
bert_embeddings = []
for sent in sentences:
 outputs = bert(tokenizer(sent, return_tensors="pt"))
```


continued...

Neural Encoding with Language Models

```
 # Use layer 8 (found to correlate best with semantic areas)
 bert_embeddings.append(outputs.hidden_states[8].mean(dim=1))

# 3. Train encoding model: BERT → Brain
from sklearn.linear_model import Ridge
encoder = Ridge().fit(bert_embeddings[:80], brain_activity[:80])

# 4. Predict brain activity for new sentences
predictions = encoder.predict(bert_embeddings[80:])
correlation = np.corrcoef(predictions.flat, brain_activity[80:].flat)[0,1]
# Correlation ~ 0.3-0.5 in language areas (significant!)
```

Key finding: BERT layer 8 best predicts semantic areas; layers 2-4 predict phonological areas

Reference: Caucheteux & King (2022) - "Brains and algorithms partially converge"

Winter 2026

...continued

Discussion: What Does the Model "Understand"?

Does BERT understand language?

Evidence FOR understanding:

- Captures syntax and semantics
- Resolves ambiguity
- Handles long-range dependencies
- Generalizes to new examples
- Predicts brain activity
- Solves complex tasks

"If it acts like it understands, maybe it does?"

Evidence AGAINST understanding:

- No grounding in physical world
- No sensory experience
- No social context
- Brittle to adversarial examples
- No common sense reasoning
- Only pattern matching?

"Understanding requires more than statistical patterns"

Key Questions

Adversarial Examples and Brittleness

BERT can be fooled easily

```
from transformers import pipeline
classifier = pipeline("sentiment-analysis")

# Works correctly
classifier("This movie was absolutely wonderful!")
# → [{'label': 'POSITIVE', 'score': 0.9998}]

# Adding irrelevant negative words flips prediction!
classifier("This movie was absolutely wonderful! [SEP] bad bad bad bad")
# → [{'label': 'NEGATIVE', 'score': 0.9234}] # WRONG!

# Synonym substitution can break it
classifier("The food was good") # → POSITIVE (0.99)
classifier("The food was fine") # → POSITIVE (0.72) # Less confident
classifier("The food was ok") # → NEGATIVE (0.51) # WRONG!

# Typos cause problems
```


continued...

Adversarial Examples and Brittleness

```
classifier("This is amazign!") # Might work
classifier("Thsi si amzaign!") # Likely wrong prediction
```

Implications for Deployment

- **Adversarial attacks**: Malicious users can manipulate predictions
- **Robustness testing**: Always test with perturbed inputs
- **Defense strategies**: Adversarial training, input validation, ensemble methods

Key insight: Models learn statistical patterns, which can include spurious correlations

Winter 2026

...continued

2. **No True Understanding**

- Pattern matching vs. comprehension
- Lack of common sense
- No world model

3. **Data Efficiency**

- Requires massive training data

Bias in Language Models

Models reflect and can amplify societal biases

```python
from transformers import pipeline
unmasker = pipeline("fill-mask", model="bert-base-uncased")

# Gender bias in occupations
unmasker("The doctor said [MASK] would be late.")
# → [('he', 0.62), ('she', 0.18), ('it', 0.08), ...]

unmasker("The nurse said [MASK] would be late.")
# → [('she', 0.71), ('he', 0.15), ('it', 0.06), ...]

# Racial bias (different sentiment for names)
classifier = pipeline("sentiment-analysis")
classifier("Emily is a brilliant scientist.") # POSITIVE: 0.98
classifier("Jamal is a brilliant scientist.") # POSITIVE: 0.94 # Lower!

# Where does bias come from?
# Training data (books, Wikipedia) contains historical biases
```


continued...

Bias in Language Models

```
18 # Model learns and sometimes amplifies these patterns
```

Mitigation Strategies

1. **Data-level:** Balanced training corpora, counterfactual augmentation
2. **Model-level:** Debiasing loss functions, fine-tuning on balanced data
3. **Output-level:** Post-hoc filtering, human review for sensitive applications
4. **Evaluation:** Regular bias audits using standardized benchmarks (WinoBias, etc.)

Winter 2026

...continued

2. **Fine-tuning Best Practices**

- Use small learning rate (1e-5 to 5e-5)
- Add warmup steps
- Monitor for overfitting
- Freeze early layers if data is limited

3. **Computational Efficiency**

- Use mixed precision training (FP16)
- Gradient accumulation for larger batch sizes
- Consider DistilBERT for faster inference
- Use FlashAttention when available

Deployment Considerations

Moving from research to production

```
# Example: Optimizing BERT for production deployment
from transformers import BertModel, BertTokenizer
import torch
import onnxruntime

# Step 1: Load model
model = BertModel.from_pretrained("bert-base-uncased")
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

# Step 2: Quantize for speed (INT8 instead of FP32)
quantized_model = torch.quantization.quantize_dynamic(
 model, {torch.nn.Linear}, dtype=torch.qint8
)
# Result: 4x smaller, 2x faster on CPU

# Step 3: Export to ONNX for production
```


continued...

Deployment Considerations

```
dummy_input = tokenizer("Hello world", return_tensors="pt")
torch.onnx.export(model, dummy_input, "bert.onnx")

# Step 4: Use ONNX Runtime for inference
session = onnxruntime.InferenceSession("bert.onnx")
# 1.5x faster than PyTorch, works on any platform
```

Optimization	Size	Latency	Quality
Original (FP32)	420MB	50ms	100%
Quantized			

...continued

- Vision + Language (CLIP, DALL-E)
- Audio + Language (Whisper)
- Grounded understanding

3. **Better Pre-training**

- More efficient objectives
- Curriculum learning
- Continual learning

4. **Smaller, More Efficient Models**

- Better compression techniques

Encoder vs Decoder Models Revisited

Different models for different tasks

p{4.5cm}}	Encoder (BERT)	Decoder (GPT)
• Classification		
• NER, QA		
• Similarity	Generation tasks:	
• Text		

- How useful are these comparisons?
- What can neuroscience learn from AI?
- What can AI learn from neuroscience?

3. **Bias and Fairness:**

- Who is responsible for addressing bias?
- Can we ever have completely unbiased models?

- Load pre-trained BERT
- Fine-tune on sentiment analysis
- Compare to baseline models

3. **Analyze Contextual Embeddings**

- Extract embeddings for polysemous words
- Visualize how context changes representations
- Compare BERT vs Word2Vec

4. **Explore Different Architectures**

- Self-attention, multi-head attention
- Positional encoding, layer norm, residuals
- Encoder-only (BERT), Decoder-only (GPT), Both (T5)

3. **BERT & Variants**

- Masked Language Modeling
- Pre-train then fine-tune paradigm
- RoBERTa, ALBERT, DistilBERT, ELECTRA

4. **Applications**

- Sanh et al. (2019) - DistilBERT
- Dao et al. (2022) - FlashAttention

Cognitive Neuroscience:

- Hagoort & Indefrey (2014) - The neurobiology of language beyond single words
- Kuperberg & Jaeger (2016) - What do we mean by prediction in language comprehension?
- Willems et al. (2016) - Prediction during natural language comprehension

- **Week 7:** Decoder models and text generation (GPT family)
- **Week 8:** Scaling laws and large language models
- **Week 9:** Prompting, in-context learning, and instruction tuning
- **Week 10:** Alignment, RLHF, and ethical considerations

The Journey Continues:

- From understanding (BERT) to generation (GPT)
- From supervised learning to few-shot learning
- From narrow tasks to general-purpose models

Questions?

Discussion Time

Topics to discuss:

- BERT applications
- Understanding vs. pattern matching
- Cognitive neuroscience connections
- Limitations and future work
- Assignment 4 questions

Thank you!

See you in Week 7 for GPT and text generation!